Лабораторная работа 4

Клименко В.А.

Импорт и загрузка данных

```
import os
import glob
import time

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.svm import LinearSVC

plt.rcParams["font.sans-serif"] = ["Arial", "DejaVu Sans", "Liberation Sans"]
sns.set_theme(style="whitegrid")

BASE_DIR = "./lab4_data/"
texts = []
labels = []

# тексты по медицине (метка 0)
for filepath in glob.glob(os.path.join(BASE_DIR, "medicine/*.txt")):
    with open(filepath, "r", encoding="utf-8") as f:
        texts.append(f.read())
        labels.append(0)

# тексты по IT (метка 1)
for filepath in glob.glob(os.path.join(BASE_DIR, "it/*.txt")):
    with open(filepath, "r", encoding="utf-8") as f:
        texts.append(f.read())
        labels.append(1)

print(f"  Загружено {len(texts)} текстов.")
print(f"Распределение классов:{pd.Series(labels).value_counts().to_dict()}")
```

```
  Загружено 60 текстов.
Распределение классов:{0: 30, 1: 30}
```

Предобработка и векторизация

```
import nltk
import re
from nltk.corpus import stopwords

nltk.download("stopwords", quiet=True)
stop_words = set(stopwords.words("russian")) | set(stopwords.words("english"))


def preprocess(text):
    text = text.lower()
    text = re.sub(r"[^а-яёa-z\s]", "", text)

    tokens = [
        word
        for word in text.split()
        if word not in stop_words and len(word) > 1
    ]
    return " ".join(tokens)


texts_clean = [preprocess(t) for t in texts]

X_train, X_test, y_train, y_test = train_test_split(
    texts_clean,
    labels,
    test_size=0.2,
    random_state=42,
    stratify=labels,
)

vectorizer = TfidfVectorizer(
    max_df=0.9, min_df=2
)  # Игнорируем слишком частые и редкие слова
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
print(f"Размерность признакового пространства: {X_train_vec.shape[1]} слов")
```

```
Размерность признакового пространства: 118 слов
```

Функция для обучения и оценки (чтобы не дублировать код)

```
def evaluate_model(model, X_train, y_train, X_test, y_test, model_name):
    start_time = time.time()
    model.fit(X_train, y_train)
    train_time = time.time() - start_time

    start_time = time.time()
    y_pred = model.predict(X_test)
    predict_time = time.time() - start_time

    acc = accuracy_score(y_test, y_pred)
    return {
        "Модель": model_name,
        "Accuracy": round(acc, 4),
        "Время обучения (с)": round(train_time, 4),
        "Время предсказания (с)": round(predict_time, 4),
        "Model_Obj": model,  # Сохраняем объект для дальнейшего анализа
    }


print("  Функция оценки готова")
```

```
  Функция оценки готова
```

Запуск всех трех алгоритмов

```
# 1. k-NN (KNeighborsClassifier, n_neighbors=5)
# 2. Дерево решений (DecisionTreeClassifier, max_depth=5, random_state=42)
# 3. Линейный SVM (LinearSVC, random_state=42, max_iter=2000)
results = []

models = [
    (KNeighborsClassifier(n_neighbors=5), "k-NN"),
    (DecisionTreeClassifier(max_depth=5, random_state=42), "Дерево решений"),
    (LinearSVC(random_state=42, max_iter=2000), "Linear SVM"),
]

for model, name in models:
    res = evaluate_model(model, X_train_vec, y_train, X_test_vec, y_test, name)
    results.append(res)

# Превращаем результаты в красивую таблицу
df_results = pd.DataFrame(results).drop(columns=["Model_Obj"])
print("  СРАВНЕНИЕ МОДЕЛЕЙ:")
display(df_results.style.highlight_max(subset=["Accuracy"], color="green"))
```

```
  СРАВНЕНИЕ МОДЕЛЕЙ:
```

	Модель	Accuracy	Время обучения (с)	Время предсказания (с)
0	k-NN	1.000000	0.001000	0.001500
1	Дерево решений	1.000000	0.001700	0.000500
2	Linear SVM	1.000000	0.001200	0.000200

Визуализация сравнения

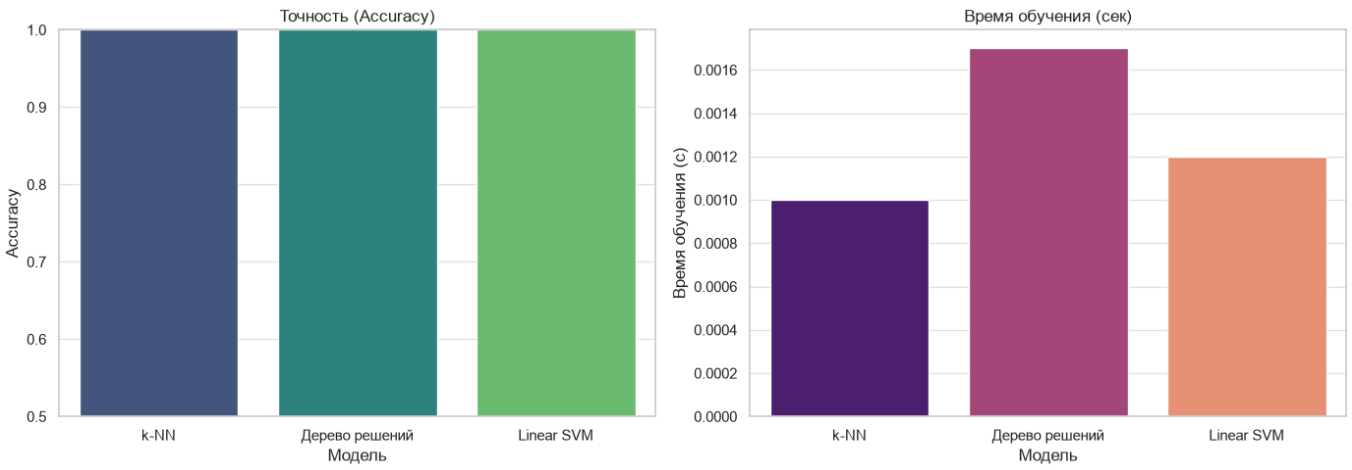
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.barplot(
    data=df_results,
    hue="Модель",
    x="Модель",
    y="Accuracy",
    ax=axes[0],
    palette="viridis",
    legend=False,
)

axes[0].set_title("Точность (Accuracy)")
axes[0].set_ylim(0.5, 1.0)

sns.barplot(
    data=df_results,
    hue="Модель",
    x="Модель",
    y="Время обучения (с)",
    ax=axes[1],
    palette="magma",
    legend=False,
)
axes[1].set_title("Время обучения (сек)")

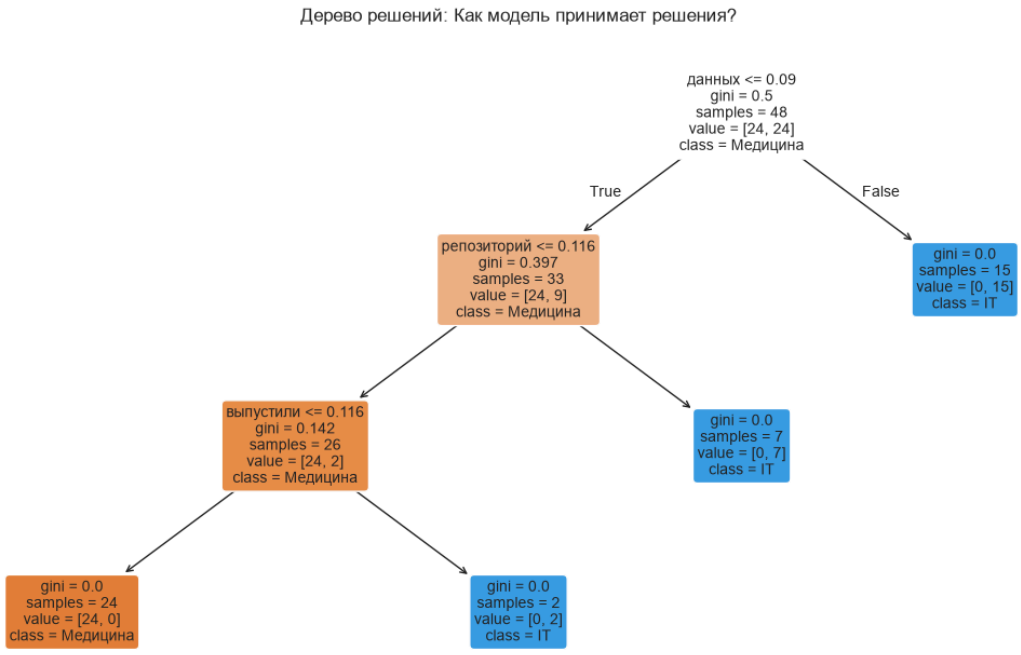
plt.tight_layout()
plt.show()
```



Интерпретация Дерева решений

```
tree_model = [
    r["Model_Obj"] for r in results if r["Модель"] == "Дерево решений"
][0]

plt.figure(figsize=(16, 8))
# Используем plot_tree из sklearn
plot_tree(
    tree_model,
    feature_names=vectorizer.get_feature_names_out(),
    class_names=["Медицина", "IT"],
    filled=True,
    rounded=True,
    fontsize=10,
    max_depth=3,  # Ограничиваем глубину для читаемости
)
plt.title("Дерево решений: Как модель принимает решения?")
plt.show()
```



Важность признаков (Feature Importances)

```
importances = tree_model.feature_importances_
feature_names = vectorizer.get_feature_names_out()

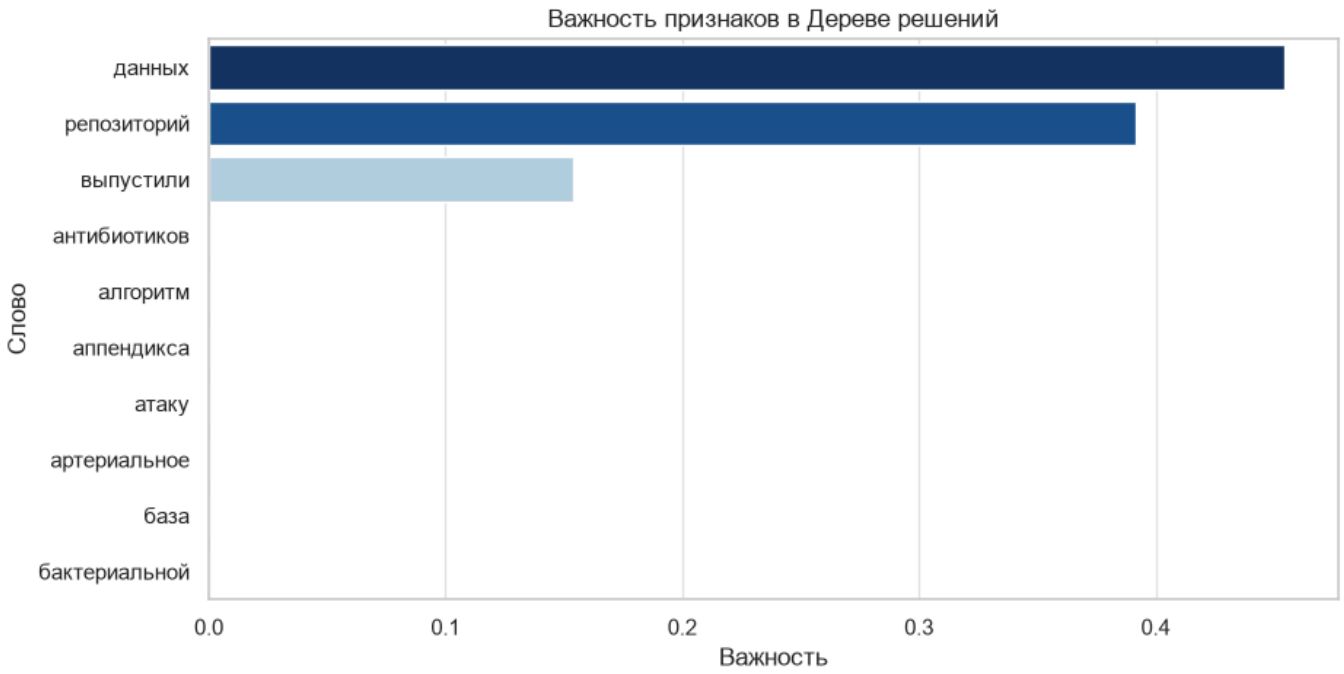
# Создаем DataFrame и сортируем
feat_df = pd.DataFrame({"Слово": feature_names, "Важность": importances})
feat_df = feat_df.sort_values(by="Важность", ascending=False).head(10)

print("  Топ-10 самых важных слов для Дерева решений:")
display(feat_df)

# Визуализация
plt.figure(figsize=(10, 5))
sns.barplot(
    data=feat_df,
    hue="Важность",
    x="Важность",
    y="Слово",
    palette="Blues",
    legend=False,
)
plt.title("Важность признаков в Дереве решений")
plt.show()
```

```
  Топ-10 самых важных слов для Дерева решений:
```

	Слово	Важность
20	данных	0.454545
85	репозиторий	0.391608
16	выпустили	0.153846
2	антибиотиков	0.000000
0	алгоритм	0.000000
3	аппендикса	0.000000
5	атаку	0.000000
4	артериальное	0.000000
6	база	0.000000
7	бактериальной	0.000000


Важность признаков в Дереве решений
данных
репозиторий
выпустили
антибиотиков
алгоритм
аппендикса
атаку
артериальное
база
бактериальной
Слово
0.0
0.1
0.2
0.3
0.4
Важность

Классифицируйте 3 тестовых файла из корня lab4_data/

```
test_files = ["test_1_it.txt", "test_2_med.txt", "test_3_noise.txt"]
print("  ТЕСТИРОВАНИЕ НА СЛОЖНЫХ ПРИМЕРАХ:")

svm_model = [r["Model_Obj"] for r in results if r["Модель"] == "Linear SVM"][0]

for f_name in test_files:
    with open(os.path.join(BASE_DIR, f_name), "r", encoding="utf-8") as f:
        raw_text = f.read()
    # Предобработка и векторизация (ВАЖНО: только transform!)
    clean_text = preprocess(raw_text)
    vec_text = vectorizer.transform([clean_text])

    # Предсказание (возьмем, например, SVM)
    pred = svm_model.predict(vec_text)[0]
    class_name = "IT" if pred == 1 else "Медицина"

    print(f"\nФайл: {f_name}")
    print(f"Текст: {raw_text[:80]}...")
    print(f"  Предсказание: {class_name}")
```

```
  ТЕСТИРОВАНИЕ НА СЛОЖНЫХ ПРИМЕРАХ:

Файл: test_1_it.txt
Текст: Программа для диагностики заболеваний использует алгоритмы машинного обучения дл...
  Предсказание: Медицина

Файл: test_2_med.txt
Текст: В больнице установили новые компьютеры, но врачи жалуются, что медицинская инфор...
  Предсказание: Медицина

Файл: test_3_noise.txt
Текст: Сегодня отличная погода, я пошел гулять в парк и купил мороженое....
  Предсказание: Медицина
```

Заполните выводы своими словами

```
svm_model = [r["Model_Obj"] for r in results if r["Модель"] == "Linear SVM"][0]

with open(
    os.path.join(BASE_DIR, "test_3_noise.txt"), "r", encoding="utf-8"
) as f:
    raw_text = f.read()
clean_text = preprocess(raw_text)
vec_text = vectorizer.transform([clean_text])
pred = svm_model.predict(vec_text)[0]
class_name = "IT" if pred == 1 else "Медицина"

print("Какая модель показала наилучшую Accuracy?")
```

```
display(df_results.style.highlight_max(subset=["Accuracy"], color="green"))
print("-" * 85)

print("\nКакая модель обучалась быстрее всего?")
display(
    df_results.style.highlight_min(subset=["Время обучения (с)"], color="blue")
)

print("Быстрее всего обучается k-NN, так как это ленивый алгоритм. ")
print(
    "Он просто запоминает выборку а все вычисления производит на этапе предсказания."
)
print("-" * 85)

print("\nКакое слово оказалось самым важным для Дерева решений?")
display(feat_df.head(5))
print("-" * 85)

print(
    "\nКак модель классифицировала 'test_3_noise.txt' (про погоду) и почему это
произошло?"
)
print(f"\nТекст: {raw_text[:80]}...")
print(
    f"  Предсказание: {class_name}, так как в бинарной классификации нет варианта
'другое'."
)
print("-" * 85)

print("\nПочему для текстов мы используем LinearSVC, а не SVM с RBF-ядром?")
print(
    "\nПри векторизации текста (TF-IDF) получается пространство, в котором данные линейно
разделимы."
)
print(
    "Использование сложного RBF-ядра замедлит работу и может привести к переобучению."
)
print("-" * 85)
```

```
Какая модель показала наилучшую Accuracy?
```

	Модель	Accuracy	Время обучения (с)	Время предсказания (с)
0	k-NN	1.000000	0.001000	0.001500
1	Дерево решений	1.000000	0.001700	0.000500
2	Linear SVM	1.000000	0.001200	0.000200

```
-------------------------------------------------------------------------------------
```

```
Какая модель обучалась быстрее всего?
```

	Модель	Accuracy	Время обучения (с)	Время предсказания (с)
0	k-NN	1.000000	0.001000	0.001500
1	Дерево решений	1.000000	0.001700	0.000500
2	Linear SVM	1.000000	0.001200	0.000200

```
Быстрее всего обучается k-NN, так как это ленивый алгоритм.
Он просто запоминает выборку а все вычисления производит на этапе предсказания.
--------------------------------------------------------------------------------

Какое слово оказалось самым важным для Дерева решений?
```

	Слово	Важность
20	данных	0.454545
85	репозиторий	0.391608
16	выпустили	0.153846
2	антибиотиков	0.000000
0	алгоритм	0.000000

```
--------------------------------------------------------------------------------

Как модель классифицировала 'test_3_noise.txt' (про погоду) и почему это произошло?

Текст: Сегодня отличная погода, я пошел гулять в парк и купил мороженое....
  Предсказание: Медицина, так как в бинарной классификации нет варианта 'другое'.
--------------------------------------------------------------------------------

Почему для текстов мы используем LinearSVC, а не SVM с RBF-ядром?

При векторизации текста (TF-IDF) получается пространство, в котором данные линейно
разделимы.
Использование сложного RBF-ядра замедлит работу и может привести к переобучению.
--------------------------------------------------------------------------------
```